MemCatalog: A Self-Evolving Memory Management System for LLM Agents

Anonymous Author(s)

Abstract

LLM agents rely on persistent memory to retrieve information from past conversations. Existing memory systems primarily manage stored facts and optimize retrieval, but provide limited support for learning from previous memory query executions. We present MemCatalog, a self-evolving memory management system that continually updates stored information and improves query processing using historical execution feedback. Realizing this design raises two technical challenges. First, memory organization should capture not only stored facts but also how the system retrieves and uses them across queries. MemCatalog introduces a three-level memory architecture that organizes conversation history, extracted facts, and memory query history. It tracks fact versions and supporting evidence to maintain accurate memory-use attribution and provide reusable execution feedback. Second, query processing should accommodate different evidence requirements while balancing retrieval effectiveness and execution cost. MemCatalog combines query planning with cost-aware execution to handle single-fact retrieval, independent composition, and dependent composition. It exploits current evidence and historical execution feedback to reuse verified results and adapt execution efforts to queries. Across eight long-term memory benchmarks, MemCatalog matches or outperforms the strongest evaluated prior system on every benchmark, with an average gain of 2.6 points. On MemBench, it achieves higher accuracy with 50.7% fewer memory-side LLM calls than a fixed three-call budget.

Keywords

Agent memory management, self-evolving memory, memory query history, query planning, cost-aware execution

1 Introduction

LLM agents increasingly interact with users across multiple sessions, relying on past preferences and experiences to provide context-aware responses [14, 16, 29]. *Memory* is the persistent data underlying these interactions, which retains information from past interactions and serves queries issued by the agent. As interaction histories grow, effective memory management requires organizing and maintaining stored information while efficiently processing memory queries from the agent.

Both the AI and database communities have made progress in *memory organization* and *query processing* for LLM agents. In the AI community, MemGPT manages context hierarchies [16], Mem0 extracts and updates facts [4], and Zep and A-MEM organize temporal relationships and linked notes [20, 31]. RMM, MemRL, and Live-Evo learn retrieval priorities or experience utilities from feedback [24, 34, 35], while MemR^3 adapts retrieval through reflective control [5]. In the database community, VikingMem manages events and entities through selective extraction, state updates, and temporal consolidation [6]. MemLens estimates the marginal contribution of memory units to response quality and uses these estimates for retention and retrieval [26].

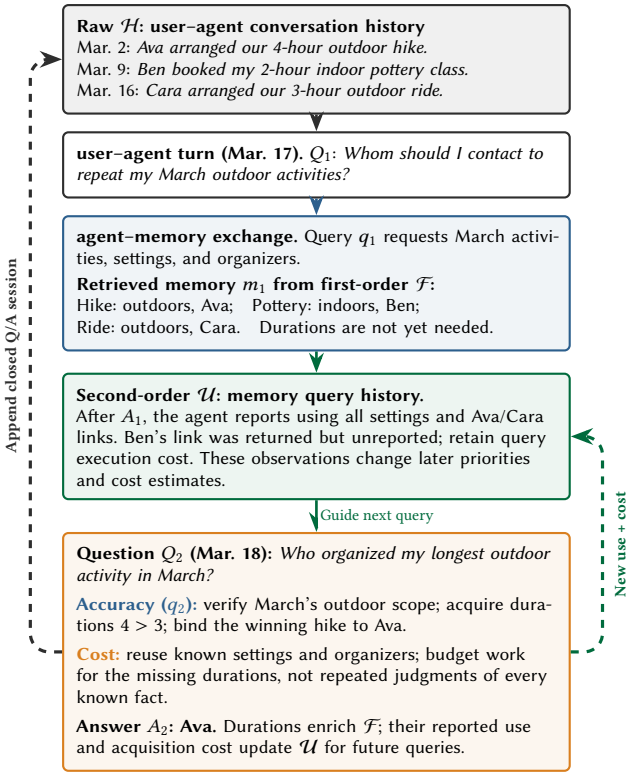


Figure 1: Conversation history and memory query history across two queries. The gray loop retains completed conversations; the green loop retains use and cost observations for subsequent queries. Illustrative example.

Limitations. Despite these advances, existing memory systems provide *limited support for continually improving memory management based on previous query executions.* As summarized in Table 1, this limitation manifests in two aspects.

First, *memory organization* primarily focuses on user knowledge without systematically retaining memory-use experience. Consider Alice's March activities in Figure 1: a hike organized by Ava, a pottery class organized by Ben, and a ride organized by Cara. For *Whom should I contact to repeat my March outdoor activities?*, memory retrieves whether each activity was indoor or outdoor, along with its organizer. The agent identifies the hike and ride as outdoor activities and returns Ava and Cara, excluding the indoor pottery class. However, storing these facts alone does not capture which information contributed to the answer or the cost of retrieving it. Consequently, subsequent queries cannot reuse this experience to prioritize useful evidence or avoid unnecessary retrieval.

Table 1: Memory organization and query processing in representative memory systems. Memory organization covers conversation histories, extracted facts, and memory-operation records. Query processing includes query-aware, cost-aware, and history-aware optimization. Query-aware optimization adapts execution to query requirements; cost-aware optimization selects execution strategies based on estimated costs; history-aware optimization uses past agent–memory interactions to guide subsequent queries. ✓: explicitly described; ×: not described for that purpose.

System	Memory Organization			Query Processing		
	user–agent interaction	Extracted facts	agent–memory interaction	Query-aware optimization	Cost-aware optimization	History-aware optimization
VikingMem [6]	×	✓	×	×	×	×
MemLens [26]	✓	✓	×	×	×	×
MemGPT [16]	✓	✓	×	✓	×	×
Mem0 / Mem0^g [4]	×	✓	×	×	×	×
Zep [20]	✓	✓	×	×	×	×
MemoryBank [36]	✓	✓	×	×	×	×
A-MEM [31]	✓	✓	×	×	×	×
MemR^3 (Zep) [5]	✓	✓	×	✓	×	×
MemCatalog	✓	✓	✓	✓	✓	✓

Second, *query processing* primarily focuses on retrieving relevant facts without explicitly handling query-specific computations. For example, finding an activity organizer requires a single fact, counting outdoor activities requires examining all activities, and finding the organizer of the longest outdoor activity requires comparing durations before identifying the organizer. A generic retrieval workflow may return relevant facts without completing these computations. Moreover, without considering previous execution experience, the system may repeat completed work or waste computation on already verified facts.

Our Proposal. We propose MemCatalog, a *self-evolving memory management system* for LLM agents. The key idea is to jointly manage conversation history and memory query history, using execution feedback from past queries to improve subsequent query processing. Realizing this design raises two technical challenges.

Challenge C1: Accurate Memory-Use Attribution. The first challenge is to maintain query execution history by accurately attributing query results to their supporting facts. For example, Alice's contact query returns Ava and Cara, but the indoor pottery class is also needed to exclude Ben. Recording all retrieved facts includes irrelevant evidence, while tracking only facts mentioned in the answer misses evidence used for exclusion.

To address this challenge, MemCatalog introduces a *three-level memory architecture* (Section 3). Raw memory retains conversation history, first-order memory stores facts extracted from conversations, and second-order memory maintains memory query history. MemCatalog tracks the sources and versions of retrieved facts, together with their reported use, execution outcomes, and costs. For derived results, derivation links identify supporting facts and propagate usage attribution to these facts. The resulting query execution history provides reusable feedback for subsequent query processing.

Challenge C2: Adaptive Memory Query Processing. The second challenge is to construct and execute query plans that accommodate different evidence requirements while exploiting historical execution feedback. For example, Alice's longest-activity query requires comparing activity durations before retrieving the corresponding organizer. Previously verified facts can be reused, while missing information requires additional retrieval.

To address this challenge, MemCatalog combines *query planning* (Section 4) with *cost-aware execution* (Section 5). MemCatalog constructs query plans for single-fact retrieval, independent composition, and dependent composition, explicitly representing their evidence requirements. Using the current evidence state and historical execution feedback, MemCatalog reuses verified facts, allocates query-specific execution budgets, and adaptively selects further retrieval operations. Execution outcomes and costs are recorded in second-order memory, enabling subsequent queries to benefit from accumulated experience.

Contributions. Our contributions are summarized as follows.

- **Self-evolving memory management.** We introduce MemCatalog, a memory management system that continually evolves both stored information and query processing by retaining conversation history and memory query history, and using historical execution feedback to improve subsequent queries.
- **Memory organization and query processing.** We develop a three-level memory architecture that maintains conversation history, extracted facts, and memory query history with source and version tracking. We further propose query planning and cost-aware execution that handle different evidence requirements, reuse verified facts, and adapt execution effort based on historical feedback.
- **Experimental evaluation.** Across eight long-term memory benchmarks, MemCatalog matches or outperforms the strongest evaluated prior system on every benchmark, as shown in Figure 2, with an average improvement of 2.6 points. On MemBench, it achieves higher accuracy with 50.7% fewer memory-side LLM calls than a fixed three-call budget. Experiments with eight LLM backbones further evaluate its generality.

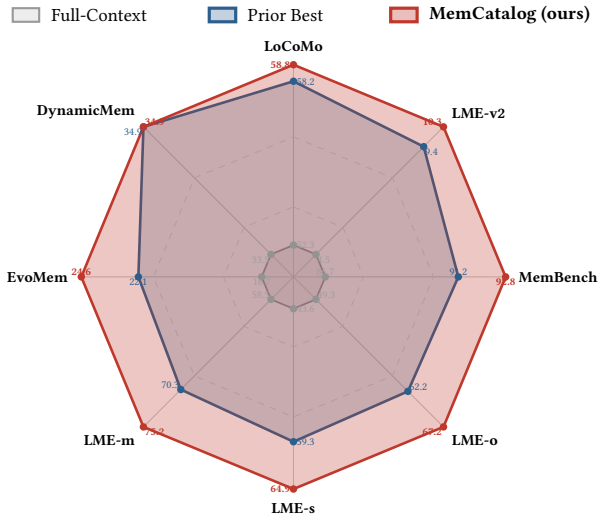


Figure 2: Benchmark scores, normalized separately on each axis between Full-Context (inner ring) and MemCatalog (outer ring). Labels give raw scores; Prior Best is the strongest evaluated baseline per benchmark (Table 4).

2 Preliminaries

The user exchanges a *question* Q and an *answer* A with the agent. The agent exchanges a *memory query* q and *retrieved memory* m with the memory system:

$$\text{User} \xrightarrow{Q} \text{Agent} \xrightarrow{q} \mathcal{M} \xrightarrow{m} \text{Agent} \xrightarrow{A} \text{User}. \tag{1}$$

A question can induce several memory queries; its conversation history and memory query history are kept separately. The latter records queries, returned facts or results, reported use, and execution outcomes and costs. The agent uses m to generate A. The support semantics apply to m; claims the agent adds to A require separate justification.

A memory query specifies an information need, an entity or population scope, and a temporal scope. Execution uses the source snapshot available when q arrives; newly closed sessions become available to later queries. Current and historical questions can select different versions. We consider finite fact requests and computations over that snapshot.

Retrieved memory is $m = (v, E, \sigma)$, where v is a value, fact collection, justified partial bound, or $\perp$ when no value can be established. E is a finite evidence bundle of source-backed fact versions and evidence checks. A check tests a condition required for the result, such as source support, temporal version, population coverage, or request dependency. σ is `supported`, `partial`, or `unknown`. A count needs evidence that covers its requested population; a dependent result needs evidence for each required link. Supported means all required checks are discharged; partial means some evidence is established but a required check remains unresolved; unknown means no source support for the result is established. A verified empty scope establishes supported absence. An unresolved check leaves its predicate unknown. These states are defined by the required evidence checks of a validated plan against its source snapshot.

Table 2: Core notation.

Symbol	Meaning
Q, A	User question Q; agent answer A
q, m	Memory query q; retrieved memory m
$\mathcal{H}, \mathcal{F}, \mathcal{U}$	Conversation history $\mathcal{H}$; extracted facts $\mathcal{F}$; memory query history $\mathcal{U}$
$\mathcal{M}$	User's memory state $\mathcal{M} = (\mathcal{H}, \mathcal{F}, \mathcal{U})$
f	A first-order fact f, with source and time
P	Query plan $P = (R, D, g)$: requests R, dependencies D, result operation g
E^r, E^u	Returned fact-version IDs E^r; validated reported-use IDs $E^u \subseteq E^r$
ℓ, C	Unresolved-check fraction ℓ; memory-query LLM-call cost $C(q)$
$K(q)$	Current additional-call reservation $K(q)$ for query q
λ	Evidence-resolution/call-cost tradeoff weight λ

A user's workload is the sequence of memory queries received over time. Population coverage means that every candidate in a declared scope has been enumerated; a binding passes one request's result to another's input. Memory query history is retained per user, so later queries benefit from earlier fact use and execution outcomes without mixing users' states. Table 2 lists the core symbols; each operational quantity is defined where it is used.

3 Memory Organization

MemCatalog links user–agent conversations with records of memory queries, returned evidence, reported use, and execution cost. Stable fact-version identifiers connect use reports and action records to the exact facts involved. Computed results retain links to their supporting facts. This shared state guides later evidence access and execution.

3.1 Three-Level Memory Architecture

For each user, the memory state is

$$\mathcal{M} = (\mathcal{H}, \mathcal{F}, \mathcal{U}). \tag{2}$$

The progression $\mathcal{H} \to \mathcal{F} \to \mathcal{U}$ links source records, extracted facts, and memory query history. It expresses their data dependencies; execution can read all three levels. All levels are isolated by user.

Raw memory: conversation history. $\mathcal{H} = (s_1, \ldots, s_n)$ is an append-only sequence of closed, timestamped conversations, including questions Q, answers A, and speaker identities. Sessions remain available, including details that no previous query needed. Agent text retains its speaker identity: suggestions and generated claims are not independent user evidence. They require user confirmation or links to already supported assertions; storing an answer does not create new source support.

First-order memory: extracted facts. $\mathcal{F}$ contains source-backed assertion records f, normally with a (subject, predicate, object) payload. Each record represents a *fact version*: an assertion with particular temporal qualifiers, a stable ID, and source spans. Equal triples with different qualifiers remain distinct. Event identifiers connect facts about the same activity across sessions. A *derivation* records

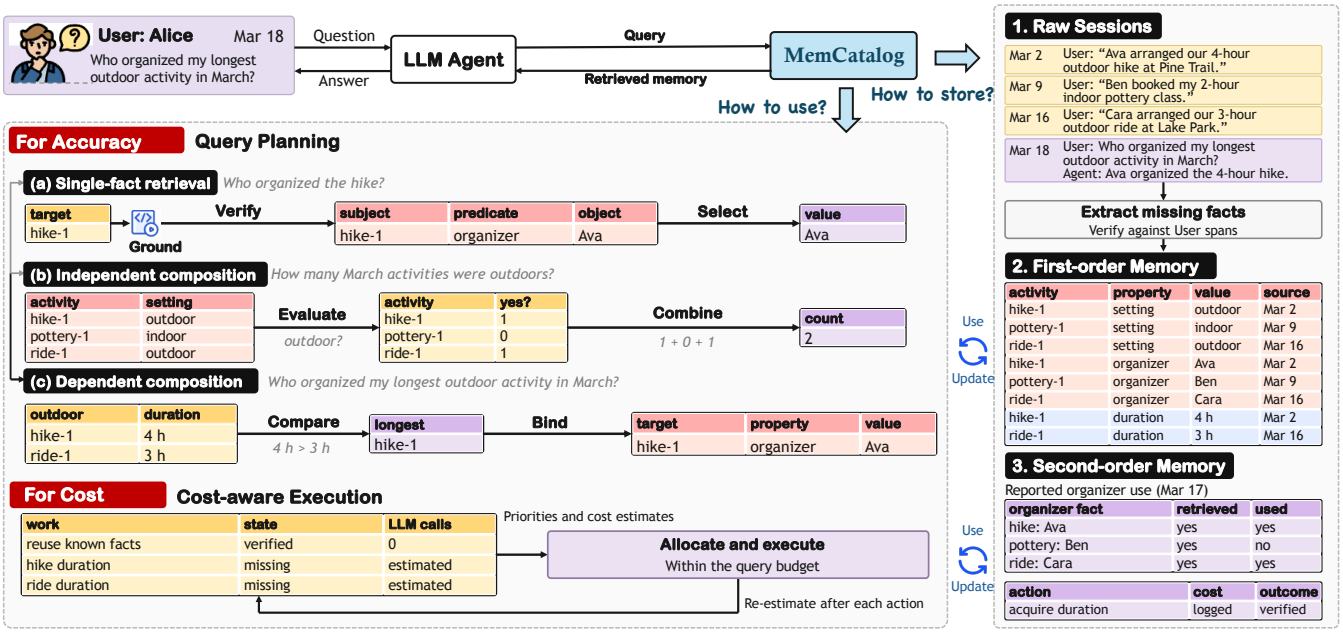


Figure 3: Memory organization and query processing. The pipelines and executor read and update the three memory levels. The example uses Alice's verified three-activity March scope. The retained state is shown after March 18: blue rows are newly acquired durations, the use report is from March 17, and the stored answer remains dialogue with its original provenance.

a computed value's input fact-version IDs and scope checks. Reuse validates these inputs before repeating computation. Extraction is query-driven: an organizer query can acquire an event's organizer, while a later comparison acquires its duration from the same source. Facts accumulate as the workload reveals what is needed.

Second-order memory: memory query history. $\mathcal{U}$ is the memory query history. It retains query context, returned evidence, support outcomes, action costs, and validated credit for agent-reported fact use. Each query record contains $(q, E^r, E^u, \sigma, C(q))$: query context, returned fact-version IDs, optional validated reported-use IDs, support status, and execution cost. Action records reference the same query. Its evidence references identify the exact first-order fact versions exposed to the agent. Reported use of a derived result expands to its supporting facts, including evidence for excluding candidates. An unreported use remains unobserved. Execution outcomes and costs are retained even for empty results or absent use reports. These records maintain two kinds of statistics: contextual fact-use counts for evidence access and estimates of evidence resolution and cost for execution. Their update rules are defined in Sections 3.2 and 4.2, and Section 5.

Raw sessions preserve unmaterialized details; facts make verified values reusable. Historical execution feedback changes their priority and the predicted cost of missing evidence, even when $\mathcal{F}$ is unchanged. Source links continue to determine evidence eligibility.

Example 1: fact acquisition and use attribution

1. Raw conversations. Alice says on March 2, *Ava arranged our 4-hour outdoor hike at Pine Trail.* On March 9, *Ben booked my 2-hour indoor pottery class.* On March 16, *Cara arranged our 3-hour outdoor ride at Lake Park.*

2. The question determines extraction. On March 17, Alice asks, *Whom should I contact to repeat my March outdoor activities?* The agent requests March activities, settings, and organizers:

Activity ID	Setting	Organizer
`hike-1`	outdoors	Ava
`pottery-1`	indoors	Ben
`ride-1`	outdoors	Cara

The six entries and verified three-activity scope support the response. For example, `(hike-1, organizer, Ava)` is one fact; unneeded durations remain in raw sessions.

3. Retrieval differs from use. The agent answers, *Ava for the hike; Cara for the ride.* The agent reports all three settings, including pottery's exclusion, and the Ava/Cara links as used, and declares the report complete. Ben's link was returned but unreported. Second-order memory records both sets of IDs, context, and cost; all facts remain available.

4. All three levels evolve. The March 18 question about the longest outdoor activity induces a memory query that acquires durations, compares $4 > 3$, and resolves its organizer, Ava. Durations enrich $\mathcal{F}$; their use and acquisition cost enrich $\mathcal{U}$. The completed question/answer session is appended to $\mathcal{H}$ when it closes.

3.2 Memory-Use Attribution

When q arrives, its source snapshot is fixed and required evidence is checked against $\mathcal{F}$. Missing properties, links, versions, or population coverage identify which source regions should be examined. Newly acquired facts enrich $\mathcal{F}$ after verification. Acquisition reconciles identities, normalizes dates and units, and retains temporal

versions. Relative dates use the source session's timestamp. Multiple valid objects can coexist; an explicit update to a single-valued property creates a new version and preserves its history.

After execution, $\mathcal{U}$ records q's context, returned fact-version IDs E^r, evidence-check outcomes, and costs. E^r identifies the facts exposed to the agent in m's evidence bundle. A use report identifies q, names the fact or derived-result IDs the agent reports using, and declares whether the report covers all memory use for that query. For a reported result, memory expands its derivation to the supporting facts, including negative evidence that excludes a candidate from a count. Every such supporting fact is included in E^r. Membership, derivation, and source/version checks against q's snapshot validate the resulting $E^u \subseteq E^r$; each fact receives at most one use credit per query, with idempotent updates by query and fact-version ID. Validation establishes the reported references and their source support; completeness is the agent's declaration. The resulting credit measures reported consumption. Only validated reports declared complete update the exposure/use statistics. Partial or missing reports leave use unobserved; execution records are retained independently.

The attribution unit is a fact version because an update can change a value's usefulness while leaving its entity and predicate unchanged. Derivation expansion recovers the filtering and exclusion evidence behind a reported result, even when the answer names only surviving candidates. Separate exposure and use counts then distinguish facts returned to the agent from facts credited by its report. These update semantics connect the three levels and provide the observations consumed by subsequent query decisions.

A closed user–agent session is appended to $\mathcal{H}$. Later queries revalidate cached scope and version checks affected by the new session; old assertions and their provenance are not erased.

Both first-order facts and second-order memory guide evidence access (Section 4) and execution (Section 5).

4 Query Planning

A memory query determines which facts should be combined and which dependencies should be resolved. The current memory state determines how much of that work is already complete. We use these two inputs together: a query-specific pipeline specifies how to obtain the required facts and combine them into a result; first-order facts supply reusable values; memory query history guides their access and the cost of unresolved work. Optimization acts on the execution of this computation: which evidence to read, which subresults to reuse, and which acquisition or verification action to perform next. Section 5 gives the selection objective and stopping rule. Semantic operators and subtask graphs provide related abstractions [9, 10, 18, 21].

4.1 Query Plans and Evidence Requirements

The three plans are *single-fact retrieval* (Lookup), *independent composition* (Independent), and *dependent composition* (Dependent). The first is the base case. The other two distinguish whether one evidence request needs a value produced by another. Independent composition includes counts, comparisons, summaries, and tuples of unrelated facts; it is not restricted to numeric aggregation. Dependent composition includes answer-dependent entity or time bindings, with independent branches allowed inside the plan.

The compiler produces $P = (R, D, g)$ for q. $R = (r_1, \ldots, r_d)$, $d \geq 1$, contains bounded lookup or population-block request templates. g is a type-checked result operation, and $(j, i, p) \in D$ binds the output of r_j to input slot p of r_i. Each template carries q's source snapshot and declared scopes. Slots accept scalar or finite-set values and have at most one binding.

Normalization exposes cross-goal bindings and removes unused requests and identity-only wrappers. Candidate enumeration, independent evaluation, and reduction form one population block; a result that determines a later request creates an explicit edge. The graph is finite and acyclic.

A plan is *atomic* when it requests one individual fact and g projects its value. Population operations remain non-atomic even with zero or one observed candidate. Thus

$$\rho(P) = \begin{cases} \textsc{Lookup}, & D = \varnothing,\ P \text{ atomic}, \\ \textsc{Independent}, & D = \varnothing,\ P \text{ non-atomic}, \\ \textsc{Dependent}, & D \neq \varnothing. \end{cases} \tag{3}$$

The partition is complete and disjoint: a plan either has a binding edge or has none; an edge-free plan is either atomic or non-atomic. Each well-formed normalized finite acyclic plan therefore selects exactly one pipeline.

This classification fixes the computation's coverage and dependency requirements. Evidence sources, access order, reuse, and execution budget remain state-dependent decisions. Across queries, a new session can enlarge a population; a verified fact can eliminate acquisition; a resolved predecessor can unlock a lookup. Query processing adapts these decisions while retaining the plan's evidence requirements.

The compiler supplies one normalized plan and validates its entity/time scopes, bindings, and g. The partition applies to that plan; different semantics-preserving representations of a question can expose different bindings. For example, a longest-activity query can request the winner's organizer after comparison or acquire every organizer before selection. Once a plan is fixed, evidence access, reuse, action ordering, and stopping are selected from the current memory state. Model-assisted compilation is charged; ambiguous scope leaves a required check unresolved.

4.2 History-Guided Evidence Access

Grounding obtains source-backed evidence for a request. All pipelines use $E_i = \text{Ground}(r_i, \mathcal{M})$ against q's fixed source snapshot. Grounding checks identity, scope, source, and version, reuses eligible facts, and acquires missing evidence from raw sessions (Section 3.2). Historical feedback orders facts and source regions without replacing these checks. Paid acquisition and semantic verification are bounded actions (Section 5).

For a fact version f and query context z, $\mathcal{U}$ retains an exposure count $n^r(f, z)$ observed under feedback declared complete and a credited-use count $n^u(f, z)$. Contexts identify topic, temporal intent, and evidence requirements. A validated report declared complete increments n^r once for each returned fact and n^u once for each fact in the validated reported-use set. Thus $0 \leq n^u \leq n^r$.

Incomplete or missing reports update neither count, preserving the distinction between unknown use and non-use. All retrieval traces and execution costs are still retained.

With positive use/non-use prior counts α_0, β_0, contextual reported-use propensity and grounding priority are

$$u(f \mid q) = \frac{\alpha_0 + n^u(f, z(q))}{\alpha_0 + \beta_0 + n^r(f, z(q))}, \qquad s(f \mid q) = \mathrm{rel}(f, q)\, u(f \mid q). \tag{4}$$

Here $\mathrm{rel}(f, q) \in [0, 1]$ is current query relevance. The ratio estimates reported use conditional on feedback declared complete. Source and version checks establish support independently. When a grounding batch has eligible alternatives, one position is reserved for a relevant, least-exposed fact in the current context, breaking ties by fact-version ID; the other positions follow s. The reserved position admits underexposed eligible evidence. Priorities affect the order of work, never the population or dependency checks required by the plan.

In Example 1, with illustrative priors $\alpha_0 = \beta_0 = 1$, the Ava/Cara organizer links have estimated use propensity 2/3 and Ben's has 1/3 in a matched contact context. This changes the priority of equally relevant facts while preserving their source-based support; an indoor-activity request can still favor Ben.

For each request, grounding first checks reusable fact versions and their required supports. A valid cache hit discharges that evidence work; a missing value, changed scope, or unresolved version leaves work for acquisition or verification. The priority in Eq. (4) orders eligible alternatives within this residual work. Population coverage and predecessor checks remain required throughout execution. This separates opportunities to reduce evidence work from the checks that establish the result.

Single-fact retrieval. For an individual target, ground its evidence and select the source-supported value at the requested time: $v = \mathrm{Select}(r, E)$. Unresolved identity, conflicting versions, or absent support leaves a gap; a missing fact remains unknown. Verified cached evidence can be returned without another semantic judgment.

Independent composition. With no cross-request bindings, ground the requests independently and apply g to their values. A tuple can combine a preference and an allergy. For a population block, establish its scope, deduplicate identities, then obtain each candidate's evidence. For the current population request r,

$$x_e = \mathrm{Eval}(r, e, E_e), \qquad v = \mathrm{Combine}(r, \{(e, x_e) : e \in \mathrm{scope}(r)\}). \tag{5}$$

x_e is a verified value or predicate outcome, or $\perp$ if unresolved. Counts use yes/no judgments; comparisons align normalized values with their candidates; summaries retain source support. Combine implements the request's count, comparison, tuple, or source-grounded synthesis; g combines request outputs.

Independent judgments may execute in parallel. A closed empty scope has a defined result, such as count 0 or no matching event. Comparisons retain all tied winners. When a later lookup consumes a winner, that binding makes the containing plan dependent and instantiates one request per bound winner, preserving ties.

An exact population result needs complete coverage and every required judgment. With h known positives and u unresolved judgments in a closed scope, return $[h, h + u]$ with partial support. Without coverage, only h is a justified lower bound. A highly ranked subset cannot establish an exact count.

Dependent composition. A node is ready only when every required predecessor has $\sigma_j = \texttt{supported}$. Evaluate ready nodes in topological order, then finalize g only when its required inputs are supported:

$$r_i^* = r_i[p \leftarrow v_j : (j, i, p) \in D], \qquad m_i = (v_i, E_i, \sigma_i) = \mathrm{Solve}(r_i^*, \mathcal{M}), \qquad v = g(v_1, \ldots, v_d). \tag{6}$$

Solve executes the normalized retrieval or independent block. Ready branches can run concurrently; a bound request returns to grounding for its newly determined target. A blocked node retains its unresolved predecessor check and waits before being passed to Solve.

Available supports yield a partial response; no established evidence yields unknown. A definitive final value is computed only from the inputs required by g. A supported predecessor with no matching target instead propagates supported absence and skips the target lookup, as specified by g; the binding is then resolved as absence.

Support invariant. The final $m = (v, E, \sigma)$ is supported only if its plan is validated and its required identity, source, version, coverage, value, binding, and result checks are discharged; E carries their supports, including the dependency ancestors of projected outputs and their selection and exclusion evidence. Otherwise the result remains partial or unknown. With correct primitive checks and scope interpretation, independent reduction and dependency-ordered binding preserve this invariant by induction. Use scores and budgets cannot close unresolved checks.

Example 2: evidence requirements of the three plans

Use Alice's three March activities from Example 1.

Lookup. *Who organized the hike?* Verify `hike-1`'s organizer and return Ava.

Independent. *How many March activities were outdoors?* Evaluate hike → yes, pottery → no, ride → yes: 1 + 0 + 1 = 2. If pottery's setting were unknown, return [2, 3].

Dependent. *Who organized my longest outdoor activity in March?* Acquire durations: hike, 4 hours; ride, 3 hours. Select `hike-1`, bind it to the organizer lookup, and return Ava. An unresolved winner cannot justify choosing Ava over Cara.

Retain new durations in $\mathcal{F}$. Reported result use credits its supporting facts, including exclusion evidence.

5 Cost-Aware Execution

The pipeline fixes the result's evidence requirements. Execution selects unresolved work and decides when to stop. A value can unlock a lookup; a discovered event can add judgments; a completed check can make another action redundant. MemCatalog therefore reselects actions after each outcome, using historical execution feedback to predict evidence benefit and cost. Predicted mean cost ranks work; maximum reservations enforce the call cap. Adaptive sampling varies inference effort across questions [1, 12]; here selection concerns the remaining evidence checks.

5.1 Estimating Evidence Benefit and Cost

An action is a bounded unit of evidence work: acquire a source-backed property, verify a disputed value, resolve a plan block, or check its result. Each pipeline supplies a finite set of bounded action templates. A template identifies required inputs, the evidence checks it targets, and its maximum call reservation. Instantiation uses the current query, candidate population, and cached supports. Valid cached support removes completed work; unresolved bindings identify prerequisites. Let $\tau(q)$ be the executed action sequence, with $\tau_j(q)$ its jth occurrence. In memory-side LLM calls,

$$C(q) = C_{\text{plan}}(q) + \sum_{j=1}^{|\tau(q)|} c(\tau_j(q)). \tag{7}$$

Every invocation is charged once, including repeated actions, acquisition, verification, and semantic synthesis or validation by g. Deterministic reads, selection, and arithmetic use zero LLM calls but can incur latency. A call that reads memory and directly produces the answer is charged once to $C(q)$ as result computation. Additional agent-only query formulation and answer rendering are recorded separately.

Measuring evidence benefit. The executor tracks the required checks for scope coverage, values, versions, bindings, and results. A check is resolved by establishing its required evidence against the current source snapshot. Deterministic consequences of newly verified values, such as a comparison, are then propagated through the plan. For the current set of instantiated checks Ω, let $\ell(\Omega)$ be the unresolved fraction, with $\ell(\varnothing) = 0$; conflicts leave a check unresolved. A validated plan and $\ell(\Omega) = 0$ establish a supported result; ambiguous compilation leaves a required check unresolved.

An action's observed reduction compares the same pre-action set: $\Delta\ell(a) = \ell_{\text{before}}(\Omega) - \ell_{\text{after}}(\Omega)$. Newly discovered candidates add checks to the next decision's set. Keeping the pre-action set fixed separates resolution of existing gaps from discovery of additional work. The executor uses the expanded set to determine support and select the next action. A scan discovering the rain-altered outing can resolve coverage while introducing an unknown setting. Measuring reduction on the original set credits the completed coverage check; the next decision's expanded set retains the new setting requirement.

Learning from comparable actions. For each executed action, $\mathcal{U}$ stores its context, pre-action check set, resolved checks, observed reduction, and actual call cost. The matching context contains action kind, plan class, unresolved-check types, candidate count, dependency depth, and cache availability (all, some, or none).

To estimate an available action's evidence resolution $\widehat{\Delta\ell}(a)$ and cost $\widehat{c}(a)$, the executor averages matched historical reductions and costs with one declared prior observation. Detailed contexts match these fields exactly. It backs off from the detailed context to the same user's action-kind/plan-class history, then to the prior. The same fields and backoff order apply to every action estimate. Estimates are restricted to the template's currently unresolved checks: benefit is zero if all its targets are supported and is bounded by their unresolved fraction in Ω. Cache availability, dependency depth, and candidate count distinguish work within the same plan class.

A cold-start template supplies the initial reduction and call-cost estimates. Its weight decreases as matched action records accumulate. Each action declares an enforced maximum reservation $c_{\max}(a)$: $0 \le c(a) \le c_{\max}(a)$, distinct from its predicted mean cost. An action that reaches its reservation returns its current evidence state.

5.2 Adaptive Selection and Refinement

Selecting ready work and its prerequisites. A lookup with an unresolved input waits for its predecessor. A sequence template lists a bounded set of prerequisite actions in dependency order, with later inputs bound to earlier outputs. Instantiation removes work already supported by cached facts and checks that the first remaining action is ready. The sequence has its own matched context; its maximum reservation covers all remaining action occurrences. Its observed gain is the reduction on the common pre-sequence check set, so a check resolved by several internal actions receives credit once. If an internal action leaves the next binding unsupported, the sequence returns its partial evidence and releases the remaining reservation. For the longest-activity organizer, duration acquisition and comparison can thus be priced together with the lookup they enable. The candidate set contains instantiated ready actions and sequence templates; newly discovered candidates trigger instantiation again.

First-order evidence identifies cache hits and unresolved checks; memory query history supplies the matched reductions and costs. From dependency-ready actions, the executor constructs a worklist $\mathcal{A}_+(q)$ by scanning candidates in descending predicted net benefit $\widehat{\Delta\ell}(a) - \lambda\widehat{c}(a)$. A positive-benefit candidate is appended when its maximum reservation, together with those already admitted, fits the remaining hard cap. The maximum reservation, rather than the predicted mean, controls admission: a cheap action on average can still require more calls than the remaining budget permits. The current additional-call budget is

$$K(q) = \sum_{a \in \mathcal{A}_+(q)} c_{\max}(a) \;\le\; K_{\max}(q) - C_{\text{spent}}(q). \tag{8}$$

Here finite $K_{\max}(q) \ge 0$ caps all memory-side calls, including planning, and C_{spent} records calls already paid. Planning is invoked only under a reservation that fits the cap; otherwise the system stops with the plan unresolved. $\lambda \ge 0$ prices evidence resolution against calls. The worklist and $K(q)$ are rebuilt after each action using the new evidence state and spent cost. Newly discovered candidates and ready lookups can therefore change both the selected work and its reservation.

When admissible actions exist, the next action maximizes predicted net benefit:

$$\begin{gathered} a^* \in \operatorname*{arg\,max}_{a \text{ admissible}} \{\widehat{\Delta\ell}(a) - \lambda\widehat{c}(a)\}, \\ \text{continue} \iff \widehat{\Delta\ell}(a^*) > \lambda\widehat{c}(a^*). \end{gathered} \tag{9}$$

If no action is admissible, execution stops. Admissibility requires available inputs and a reservation that fits the remaining cap.

Only the first selected candidate executes. The rebuilt worklist removes completed checks and reusable prerequisites before re-estimation. For example, acquiring a duration can settle the winner

Execution loop for a validated plan P

(1) Reuse valid facts and subresults; propagate their deterministic consequences and update the unresolved checks.
(2) Instantiate ready actions and bounded prerequisite sequences; estimate their remaining evidence benefit and cost from $\mathcal{U}$.
(3) Form $\mathcal{A}_+(q)$ under Eq. (8). Stop if it is empty; otherwise select a^* by Eq. (9).
(4) Execute a^* within its reservation, charging each invocation once. Retain acquired facts and record resolved checks, outcome, and actual cost.
(5) Release tentative reservations and repeat from step 1 with the updated state. Return (v, E, σ) when no further work is selected.

Figure 4: Cost-aware execution with reuse and outcome-dependent reselection.

by comparison, removing a separate winner-verification action; a conflicting duration leaves verification available.

Updating the reservation and preserving reuse. Only the selected action spends calls. Releasing its unused reservation and rebuilding the worklist lets newly ready actions compete for the remaining budget. Because actual cost is bounded by the admitted reservation, each completed action leaves total spent cost within $K_{\max}(q)$. The same test applies to a prerequisite sequence, including all of its internal calls.

Deterministic work is not repeated on unchanged inputs; semantic retries consume the finite cap. With a finite plan and snapshot, execution terminates. Disagreement can trigger verification, while unsupported evidence requires acquisition. An input change invalidates only its dependent descendants. On stopping, unresolved checks retain partial or unknown σ; they cannot become supported through budget exhaustion.

Example 3: reuse and selective acquisition

Warm query. Alice again compares the outdoor activities. Reuse verified settings and durations: $4 > 3$, with $K(q) = 0$ additional memory-side calls. Agent answer generation is separate.

New gap. A closed March 23 session says, *We planned an outdoor picnic, but rain changed our plans.* Its follow-up says, *We ate inside River Cafe instead.* The two descriptions refer to the same outing. Alice's March 24 count needs its actual occurrence and setting, not merely the planned picnic.

Selective action. Verify the outing and acquire its actual setting when predicted benefit exceeds cost, charging extraction/verification calls. Reuse the three earlier judgments; update only the new outing: $1 + 0 + 1 + 0 = 2$.

Unavailable evidence. Without the follow-up, the closed four-candidate scope supports $[2, 3]$, not an exact count. Repeating the planned-picnic fragment cannot resolve this gap.

$\mathcal{U}$ stores $C(q)$, action outcomes, and evidence resolution independently of use reports. Validated reported use updates fact priorities (Section 4.2).

6 Experiments

We evaluate the three-level memory architecture, query planning, and cost-aware execution. We examine overall accuracy, component contributions, adaptation across queries, and history-length scaling, followed by an end-to-end example.

Table 3: Evaluation benchmarks and approximate input lengths (tokens).

Benchmark	Memory ability	Input
LoCoMo [14]	Multi-session dialogue	~9K
LME-v2 [28]	Environment experience	~115K
MemBench [23]	Query planning and composition	~100K
LME-o [29]	Long-horizon retention	~40K
LME-s [29]	Long-horizon retention	~115K
LME-m [29]	Long-horizon retention	~1.5M
EvoMem [27]	Task-stream adaptation	~50K
DynamicMem [30]	Evolving user state	~127K

6.1 Experimental Setup

Datasets. Table 3 groups eight benchmarks by memory ability. LoCoMo tests multi-session dialogue [14]; LME-v2 tests environment experience from web-agent trajectories [28]. LME-o, LME-s, and LME-m vary evidence availability and history length [29]. EvoMem tests adaptation across task streams [27]; DynamicMem tracks evolving user profiles [30]. We use MemBench [23] to study single-fact retrieval, independent composition, dependent composition, and cost-aware execution. Together, these benchmarks evaluate the memory hierarchy and its accumulation of user-specific facts and memory query history over time.

Baselines. The seven baselines are Full-Context, RAG [11], MemGPT [16], Mem0 and Mem0^g [4], Zep [20], and LongMemEval-Fw [29]. They provide, respectively, full-history prompting, passage retrieval, context tiers, consolidated facts and entity relations, temporal graphs, and an indexing–retrieval–reading pipeline. We compare methods under a shared backbone and complement the overall comparison with component ablations.

Metrics and Settings. Multiple-choice tasks use exact match; open-answer tasks use F1 with benchmark-specific normalization. Comparisons are within a benchmark's metric scale. The default backbone is GPT-5.5; Section 6.3 covers four closed-source and four open-source models. Every reported call count uses $C(q)$ from Eq. (7); averages report mean $C(q)$, and totals report $\sum_q C(q)$.

6.2 Overall Accuracy

Table 4 shows that MemCatalog matches or improves the strongest evaluated baseline in every column. The mean benchmark-score margin is 2.6 points. The gains vary with the memory ability being tested.

Accuracy over long histories. The margins on LME-o, LME-s, and LME-m are 5.0, 5.6, and 4.9 points. A query over many sessions has to recover the right entities and temporal versions before computing its answer. The architecture in Section 3 separates this source history from reusable fact values; the pipelines in Section 4 then expose coverage and binding requirements.

Table 4: Answer scores across eight memory benchmarks (GPT-5.5).

Method	Recall and reasoning			Long-horizon memory			Personalized state	
	LoCoMo	LME-v2	MemBench	LME-o	LME-s	LME-m	EvoMem	DynamicMem
Prompting baselines								
Full-Context	52.3	4.5	86.7	49.3	43.6	58.5	16.7	33.5
RAG [11]	53.1	4.9	87.8	50.8	45.1	59.1	17.3	34.1
Memory systems								
MemGPT [16]	54.5	6.3	89.7	54.1	48.4	61.1	18.0	34.3
Mem0 [4]	56.4	7.2	91.2	58.3	54.2	65.2	20.4	34.5
Mem0^g [4]	57.1	7.5	90.8	60.1	56.7	67.1	21.2	34.7
Zep [20]	58.2	8.1	90.4	62.2	59.3	70.3	22.1	34.9
LongMemEval-Fw [29]	57.3	9.4	90.1	61.7	58.8	69.4	21.6	34.6
Ours								
MemCatalog	**58.8**	**10.3**	**92.8**	**67.2**	**64.9**	**75.2**	**24.6**	**34.9**

Table 5: MemBench accuracy across backbones. Best prior is the strongest of the seven baselines in Table 4, rerun with the same backbone.

Backbone	Best prior	MemCatalog
Closed-source		
GPT-5.5	91.2	**92.8**
Claude-3.5-Sonnet	87.4	**91.4**
Gemini-1.5-Pro	85.6	**89.7**
GPT-4o	82.8	**88.7**
Open-source		
DeepSeek-V4	82.7	**88.5**
Llama-3.1-70B	79.8	**86.5**
Qwen3-14B	77.4	**85.1**
Qwen3-8B	73.4	**81.3**

The result is consistent with reducing the evidence alignment left to a long generation, a difficulty observed in long-context evaluations [8, 13]. Component ablations in Section 6.6 separately test reusable facts and explicit composition.

Lookup and composition workloads. The margins are 0.6 points on LoCoMo and 1.6 on MemBench. Sections 6.4 and 6.8 examine the evidence processing and plan compilation behind these aggregate scores.

Changing user state. On EvoMem, MemCatalog gains 2.5 points over the strongest baseline; on DynamicMem, both systems score 34.9. Section 6.7 examines query positions on EvoMem; Section 6.6 tests second-order memory with unchanged aggregate $\sum_q C(q)$.

6.3 Accuracy Across Language Models

We hold MemBench and the reported configuration fixed and compare against the strongest baseline rerun with each backbone.

The gains in Table 5 range from 1.6 to 5.9 points for closed-source models and from 5.8 to 7.9 for open-source models. Their group means are 3.9 and 7.0 points. Within each group, the gain increases as the prior-baseline score falls. Each backbone receives the structured intermediate answer under the same memory configuration, with coverage, alignment, and combination handled by the pipeline before generation.

6.4 Evidence Requirements and Query Accuracy

The three pipelines differ in the evidence checks they require. Single-fact retrieval terminates once the requested value and its support are established. For population queries, independent composition also requires coverage and a supported judgment for each candidate. Dependent composition preserves the event–value bindings needed by a later selection or lookup.

Independent composition requires population coverage. A count can be wrong even when the retrieved passages are topically relevant: candidates can be omitted or duplicated, and a mentioned event may not satisfy the requested predicate. The independent-composition pipeline in Eq. (5) separates enumeration, per-candidate judgment, and tallying. In Alice's example, establishing that pottery was indoors justifies its exclusion from the outdoor count. Once each judgment is supported, arithmetic need not be another unstructured model inference.

Dependent composition preserves value–event bindings. A dependent composition should associate each normalized value with its event before ordering them. Retrieving both durations without that alignment can still select the wrong activity. When the selected activity determines an organizer request, Eq. (6) carries the required binding into the next lookup.

Table 6 makes Alice's count explicit: the three verified settings contribute $1 + 0 + 1 = 2$. The indoor setting supports exclusion and remains part of the derivation even though pottery is absent from the selected outdoor activities. If that setting is unresolved, the same closed scope yields $[2, 3]$. This example illustrates the pipeline's support requirements; the component comparisons in Section 6.6 evaluate its effect on answer scores.

These cases distinguish missing evidence from incorrect combination. A count needs both a covered scope and supported settings; an organizer selected through a comparison additionally needs the winning activity's identity to survive the lookup.

Table 6: Illustrative evaluation of Alice's March outdoor count. The closed scope contains the three activities in Example 1.

Candidate	Verified setting	Contribution
Hike	Outdoors	1
Pottery	Indoors	0
Ride	Outdoors	1
Supported count		**2**

Table 7: Budget allocation and selective refinement on MemBench.

Configuration	Average $C(q)$			$\sum_q C(q)$	Acc.
	Single-Fact Retrieval	Independent Composition	Dependent Composition		
Fixed Budget (K=1)	1.00	1.00	1.00	300	86.7
Fixed Budget (K=3)	3.00	3.00	3.00	900	89.1
MemCatalog, allocation only	1.00	3.00	3.00	532	92.8
MemCatalog	1.00	2.30	2.18	**444**	**92.8**

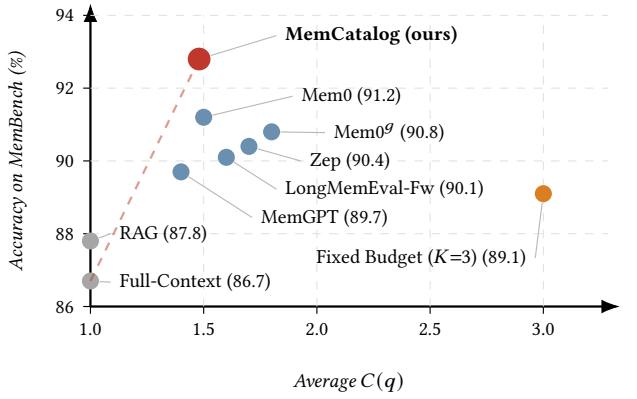


Figure 5: Accuracy and average $C(q)$ on MemBench (GPT-5.5).

6.5 Cost-Aware Execution

Table 7 separates two sources of savings in cost-aware execution: allocating effort across queries and selectively refining evidence only while it remains worthwhile.

Query-specific allocation. Fixed Budget (K=3) has $\sum_q C(q) = 900$ at 89.1 accuracy. The allocation-only configuration has $\sum_q C(q) = 532$ at 92.8: 368 lower $\sum_q C(q)$ (40.9%) with a higher score. These aggregate results favor query-specific allocation over a uniform call allowance. In the method, the reserved work depends on unresolved checks and the historical benefit and cost of available actions.

Selective refinement. Selective refinement reduces $\sum_q C(q)$ from 532 to 444, saving another 88 (16.5% of the allocation-only count), with the same reported aggregate accuracy. Average $C(q)$ is 1.00 for single-fact retrieval, 2.30 for independent composition, and 2.18 for dependent composition.

The realized costs fall below their allocated maxima. In total, $\sum_q C(q)$ falls by 456 relative to Fixed Budget (K=3), giving the reported 50.7% reduction. The allocation and refinement configurations evaluate the two controls together with the pipeline. Allocation reserves admissible work; refinement releases reservations and rebuilds the selected work after each outcome. The additional 88 saved calls quantify refinement's contribution at the same aggregate score. The reported plan-class means summarize realized cost; action selection follows the current evidence state through Eq. (9).

Figure 5 summarizes the accuracy–cost tradeoff under $C(q)$. The two reductions act at different points in execution. Allocation limits which unresolved checks receive resources; refinement revisits that choice as evidence arrives. Their combined benefit depends on retaining completed checks, so an additional call targets remaining uncertainty rather than repeating an already supported decision.

Table 8: Ablations of memory organization and query processing.

Configuration	MemBench		LME-v2	
	Score	$\sum_q C(q)$	Score	$\sum_q C(q)$
MemCatalog	**92.8**	**444**	**10.3**	**602**
Memory organization and query planning				
w/o query-plan classification	89.1	900	8.8	903
w/o composition pipelines	87.9	900	8.3	903
w/o first-order facts	86.7	900	7.6	903
w/o raw-session grounding	92.8	444	10.1	651
Second-order memory				
w/o second-order memory	92.1	444	8.6	602
w/o history-guided priority	92.5	444	9.3	602
Cost-aware execution				
w/o cost-aware allocation	92.5	756	10.3	903
w/o selective refinement	92.8	532	10.3	723

6.6 Component Ablations

Table 8 compares configurations of the memory hierarchy, query-specific pipelines, second-order memory, and cost-aware execution.

Memory organization and query computation. Removing first-order facts lowers accuracy by 6.1 points on MemBench and 2.7 on LME-v2. Removing composition pipelines lowers it by 4.9 and 2.0; removing query-plan classification lowers it by 3.7 and 1.5. Each configuration also has a larger $\sum_q C(q)$ than the full system.

These losses occur despite greater aggregate call cost. First-order facts supply reusable operands; the selected pipeline makes their composition explicit. Removing either increases repeated grounding or leaves evidence combination to generation. Raw-session grounding also preserves access to details that no earlier query materialized.

Memory query history at unchanged aggregate cost. Removing second-order memory loses 0.7 points on MemBench and 1.7 on LME-v2, while $\sum_q C(q)$ remains 444 and 602. Removing history-guided priority loses 0.3 and 1.0, again without changing those totals. The whole-history ablation removes the records used by both access priorities and action estimates; the priority ablation targets the access score in Eq. (4).

Table 9: EvoMem answer scores by position in a user's query history.

Method	Within-user question position			
	1–3	4–6	7–9	10+
EvoMem				
Full-Context	16.7	17.0	16.8	16.5
Mem0 [4]	20.1	20.5	20.4	20.2
Zep [20]	21.8	22.2	22.1	22.0
MemCatalog	**22.2**	**24.4**	**26.1**	**27.8**

Table 10: Answer accuracy and compilation cost on MemBench.

Plan compiler	$C_{\text{plan}}(q)$	Answer acc.
Rule-based (ours)	0	92.8
LLM plan compiler	1	93.2
Always Single-Fact Retrieval	0	87.5

The score differences associate historical execution feedback with better answers at unchanged aggregate expenditure. The two ablations test different uses of that state: evidence ordering and the combined access/execution policy.

Allocation and refinement. Without cost-aware allocation, $\sum_q C(q)$ increases by 312 on MemBench and 301 on LME-v2; accuracy falls by 0.3 on the former and is unchanged on the latter. Without selective refinement, $\sum_q C(q)$ increases by 88 and 121 without changing the reported scores. Allocation affects both quality and cost; selective refinement removes work after allocation.

6.7 Accuracy Across a User's Query History

Table 9 groups EvoMem answer scores by position in each user's query history. MemCatalog rises from 22.2 to 27.8 across the four buckets, while the strongest baseline remains nearly flat. These scores describe the full system as both facts and memory query history accumulate. The separate history ablations in Section 6.6 complement this trend by comparing answer scores with and without historical feedback at unchanged aggregate calls.

6.8 Query-Plan Compilation

Table 10 compares end-to-end answers under different compilers. The rule-based compiler has $C_{\text{plan}}(q) = 0$. The LLM compiler improves answer accuracy by 0.4 points. Its $C_{\text{plan}}(q) = 1$ adds 300 to $\sum_q C(q)$, or 67.6% of the full system's $\sum_q C(q) = 444$.

Always choosing Single-Fact Retrieval loses 5.3 end-to-end points. This comparison tests the value of exposing composition in the plan. Compilation fixes the computation and its evidence requirements; the access and execution decisions in Sections 4–5 determine how its evidence is obtained and reused.

6.9 Scaling with History Length

Table 12 increases history from 40K to 1.5M tokens, a 37.5× range. Average $C(q)$ remains 1.9 for MemCatalog and 1.0 for Full-Context. Yet latency grows 20.7× for Full-Context, from 2.3s to 47.6s, and only 1.09× for MemCatalog, from 3.2s to 3.5s. $C(q)$ alone therefore understates the differences in inference expense.

At 40K, structured memory access is slower by 0.9s. At 115K, MemCatalog takes 3.4s against 5.8s, and at 1.5M it is 13.6× faster. Structured access has a fixed cost at short histories; reusable facts and bounded evidence processing avoid placing the entire source history in each query as that history grows.

Full-Context's single memory-reading result call processes a longer prompt; MemCatalog's 1.9 calls use selected facts and unresolved evidence blocks. The hierarchy controls context growth, while the executor controls repeated semantic work.

Accuracy advantages over Full-Context remain 17.9, 21.3, and 16.7 points across the three variants. The pipelines retain explicit coverage and binding checks while reusing available facts; new evidence is acquired only when it is required.

6.10 Case Study

Table 11 traces three queries over Alice's illustrative data from Examples 1–3, including the persistent updates.

Stored content changes subsequent decisions. After Turn 1, $\mathcal{F}$ supplies verified settings and organizers; $\mathcal{U}$ distinguishes six returned facts from five credited by the use report. Turn 2 acquires durations and records the acquisition outcome and cost. Repeating that comparison can reuse its complete inputs with $K(q) = 0$ additional calls. The recorded action conditions also inform the cost of later acquisitions.

Memory query history changes decisions at fixed content. With Turn 1's fact values fixed, the returned/used distinction still changes access. The unit priors in Section 4.2 assign matched-contact use propensities 2/3 to Ava/Cara links and 1/3 to Ben's. Pottery's indoor setting retains credit for exclusion. Source checks determine eligibility; the learned values order equally relevant facts. Recorded outcomes inform the predicted benefit and cost of later missing-duration work; valid cached durations remove that work.

The query history distinguishes two reasons for an item to be absent from the answer. Pottery's setting supports exclusion; Ben's organizer link was returned but not reported as used. Both are absent from the final contact names, but only the setting receives use credit. Derivation links preserve this distinction when an agent reports the combined result rather than listing every supporting fact.

A changed population changes the remaining work. Turn 3 adds one outing to the March scope. The three earlier setting judgments remain reusable, while the new outing's actual setting requires verification. The follow-up establishes that it occurred indoors, giving $1 + 0 + 1 + 0 = 2$; without that evidence, the result remains $[2, 3]$. The cache removes completed checks; second-order memory guides the new setting acquisition. Population coverage still determines whether the count can be exact.

Table 11: Illustrative memory state and query execution across three turns. Retrieved memory is $m_t = (v_t, E_t, \sigma_t)$; updates persist for later queries.

Turn and question	Pipeline and required evidence	Retrieved memory	Persistent update
Turn 1 (Mar. 17) *Whom should I contact to repeat my March outdoor activities?*	**Independent composition.** Verify the closed three-activity scope, judge each setting, and project the organizers of the outdoor activities.	$v_1 = \{\text{Ava}, \text{Cara}\}$; E_1 contains the scope, three setting judgments, and three returned organizer links; $\sigma_1 =$ `supported`.	$\mathcal{H}$: closed question/answer. $\mathcal{F}$: settings and organizer links. $\mathcal{U}$: returned evidence, credited settings and Ava/Cara links, unreported Ben link, outcome, and $C(q_1)$.
Turn 2 (Mar. 18) *Who organized my longest outdoor activity in March?*	**Dependent composition.** Reuse settings, acquire the missing durations, compare $4 > 3$, and bind `hike-1` to its organizer.	$v_2 =$ Ava; E_2 contains the duration comparison and winner-to-organizer binding; $\sigma_2 =$ `supported`.	$\mathcal{H}$: closed question/answer. $\mathcal{F}$: two duration facts. $\mathcal{U}$: their use, acquisition outcome, and $C(q_2)$. A repeated comparison needs $K(q) = 0$ additional memory-side calls.
Turn 3 (Mar. 24) *How many March activities were outdoors?*	**Independent composition with selective refinement.** Reuse three completed judgments and evaluate only the rain-altered outing's actual setting.	With the follow-up, $v_3 = 1 + 0 + 1 + 0 = 2$ and $\sigma_3 =$ `supported`. Without it, $v_3 = [2, 3]$ and $\sigma_3 =$ `partial`.	$\mathcal{H}$: closed question/answer. $\mathcal{F}$: verified actual setting. $\mathcal{U}$: refinement outcome and $C(q_3)$. Earlier support is retained but is not transferred to the new outing.

Table 12: Cost, latency, and answer scores across LongMemEval history lengths.

History	Average $C(q)$	Latency (s)	Score
Full-Context			
~40K (LME-o)	1.0	2.3	49.3
~115K (LME-s)	1.0	5.8	43.6
~1.5M (LME-m)	1.0	47.6	58.5
MemCatalog			
~40K (LME-o)	1.9	3.2	67.2
~115K (LME-s)	1.9	3.4	64.9
~1.5M (LME-m)	1.9	3.5	75.2

7 Related Work

Memory management systems. Memory systems manage context tiers, conversational facts, temporal relations, and linked notes [4, 16, 20, 31]. MemoryBank models forgetting [36]; Generative Agents synthesize reflections [17]. VikingMem introduces event/entity abstractions, event-driven entity updates, timeline compression, and time-weighted retrieval [6]. Tool-call and failure histories update tool entities with experience for future invocations [6]. MemLens organizes atomic units and summaries in a provenance-linked tree. Shapley-style marginal contributions guide retention and reranking [26].

MemCatalog jointly manages conversation history and memory query history. Its second-order memory associates returned and reported-use fact versions with derivations, action outcomes, and costs. This state guides both access and execution (Table 1).

Learning from memory interactions. RMM uses memory citations to update a reranker [24]; MemRL learns episodic utilities from rewards [34]; Live-Evo updates experience weights and guidelines [35]. MemSkill learns a selection policy over memory-construction skills and revises those skills using difficult cases [33]; UMEM jointly optimizes extraction and memory updates using query-neighborhood utility [32]. MemCatalog validates reported evidence references for version-specific credit, including facts supporting a derived result. Missing reports leave use unobserved; execution outcomes still update cost estimates. Source checks determine support; feedback guides later access and execution.

Memory queries and adaptive execution. HippoRAG retrieves over a knowledge graph [7]; MemoRAG uses global memory to generate retrieval clues [19]. LongMemEval separates indexing, retrieval, and reading [29]. MemR^3 controls retrieval, reflection, and answering through an evidence-gap tracker [5]. LOTUS optimizes semantic operators [18]; DocETL rewrites document pipelines [21]; EVAPORATE and QUEST optimize extraction work [2, 22]. Our pipelines expose coverage and binding requirements; interaction history guides the evidence work remaining within them.

Eddies adapts operator routing [3], and Bao learns optimizer hints from execution feedback [15]. Here actions acquire or verify conversational evidence, and their outcomes change the next decision. Self-consistency aggregates sampled answers [25]; Adaptive-Consistency and ESC stop from answer agreement [1, 12]. Our rule prices evidence resolution against cost and preserves verified subresults across queries.

8 Conclusion

MemCatalog links conversation history, extracted facts, and memory query history in a self-evolving memory management system. Fact versions and derivations connect reported use to reusable evidence. Query plans encode coverage and dependency requirements; cost-aware execution reuses verified results and selects remaining work using historical outcomes and costs. Across eight benchmarks, it improves the strongest evaluated prior system by 2.6 points on average. On MemBench, it achieves higher accuracy with 50.7% fewer memory-side LLM calls than a fixed three-call budget.

References

[1] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 12375–12396. doi:10.18653/v1/2023.emnlp-main.761

[2] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proceedings of the VLDB Endowment* 17, 2 (2023), 92–105. doi:10.14778/3626292.3626294

[3] Ron Avnur and Joseph M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, 261–272. doi:10.1145/342009.335420

[4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025). doi:10.48550/arXiv.2504.19413

[5] Xingbo Du, Loka Li, Duzhen Zhang, and Le Song. 2025. MemR^3: Memory Retrieval via Reflective Reasoning for LLM Agents. *arXiv preprint arXiv:2512.20237* (2025). doi:10.48550/arXiv.2512.20237

[6] Jiajie Fu, Junwen Chen, Mengzhao Wang, Aoxiang He, Maojia Sheng, Xiangyu Ke, Yifan Zhu, and Yunjun Gao. 2026. VikingMem: A Memory Base Management System for Stateful LLM-based Applications. *arXiv preprint arXiv:2605.29640* (2026). doi:10.48550/arXiv.2605.29640

[7] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv preprint arXiv:2405.14831* (2024). doi:10.48550/arXiv.2405.14831

[8] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models?. In *First Conference on Language Modeling*. https://arxiv.org/abs/2404.06654

[9] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.02406

[10] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391. https://proceedings.mlr.press/v235/kim24y.html

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[12] Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. 2024. Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=ndR8Ytrzhh

[13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[14] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 13851–13870. doi:10.18653/v1/2024.acl-long.747

[15] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, 1275–1288. doi:10.1145/3448016.3452838

[16] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023). doi:10.48550/arXiv.2310.08560

[17] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 1–22. doi:10.1145/3586183.3606763

[18] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184. doi:10.14778/3749646.3749685

[19] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2024. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. *arXiv preprint arXiv:2409.05591* (2024). doi:10.48550/arXiv.2409.05591

[20] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025). doi:10.48550/arXiv.2501.13956

[21] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3035–3048. doi:10.14778/3746405.3746426

[22] Zhaoze Sun, Chengliang Chai, Qiyan Deng, Kaisen Jin, Xinyu Guo, Han Han, Ye Yuan, Guoren Wang, and Lei Cao. 2025. QUEST: Query Optimization in Unstructured Document Analysis. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4560–4573. doi:10.14778/3749646.3749713

[23] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 19336–19352. doi:10.18653/v1/2025.findings-acl.989

[24] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Rajan Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. 2025. In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 8416–8439. doi:10.18653/v1/2025.acl-long.413

[25] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2203.11171

[26] Shuyue Wei, Chang Liu, Zimu Zhou, Yongxin Tong, and Lizhen Cui. 2026. MemLens: A Value-Aware Memory Management System with Interactive Analytics for LLM-based Agents. *Proceedings of the VLDB Endowment* 19, 12 (2026), 4606–4609. https://arxiv.org/abs/2607.25992

[27] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. *arXiv preprint arXiv:2511.20857* (2025). doi:10.48550/arXiv.2511.20857

[28] Di Wu, Zixiang Ji, Asmi Kawatkar, Bryan Kwan, Jia-Chen Gu, Nanyun Peng, and Kai-Wei Chang. 2026. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues. *arXiv preprint arXiv:2605.12493* (2026). doi:10.48550/arXiv.2605.12493

[29] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*. https://arxiv.org/abs/2410.10813

[30] Wenya Xie, Shengming Zhou, Zelin Li, Pouya Parsa, Shuang Zhou, Xinheng Ding, Chinmay Arvind, Guanchu Wang, Vladimir Braverman, Ali Payani, Yantao Zheng, and Zirui Liu. 2026. DynamicMem: A Long-Horizon Memory Benchmark in Real-World Settings. *arXiv preprint arXiv:2606.22877* (2026). doi:10.48550/arXiv.2606.22877

[31] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110* (2025). doi:10.48550/arXiv.2502.12110

[32] Yongshi Ye, Hui Jiang, Feihu Jiang, Tian Lan, Yichao Du, Biao Fu, Xiaodong Shi, Qianghuai Jia, Longyue Wang, and Weihua Luo. 2026. UMEM: Unified Memory Extraction and Management Framework for Generalizable Memory. *arXiv preprint arXiv:2602.10652* (2026). doi:10.48550/arXiv.2602.10652

[33] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents. *arXiv preprint arXiv:2602.02474* (2026). doi:10.48550/arXiv.2602.02474

[34] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. 2026. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. *arXiv preprint arXiv:2601.03192* (2026). doi:10.48550/arXiv.2601.03192

[35] Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. 2026. Live-Evo: Online Evolution of Agentic Memory from Continuous Feedback. *arXiv preprint arXiv:2602.02369* (2026). doi:10.48550/arXiv.2602.02369

[36] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19724–19731. doi:10.1609/aaai.v38i17.29946